

Contents

Les flux conditionnels	3
1 Présentation des flux conditionnels	3
1.1 Objectif des structures conditionnelles	3
1.2 Logique fondamentale	3
1.3 Le mot-clé if	4
1.4 Structure if / else	4
1.5 Structure if / elif / else	5
1.6 Expression conditionnelle (opérateur ternaire)	5
1.7 Opérateurs de comparaison	6
1.8 Opérateurs logiques	6
1.9 Évaluation booléenne implicite	7
1.10 Chaînage des comparaisons	7
1.11 Court-circuit logique	8
1.12 match / case — Python 3.10+	8
Avec conditions — guards	9
1.13 Cas métier — Système de validation d'accès	9
1.14 Pièges fréquents	9
Confusion entre = et ==	9
Confusion entre & et and	10
Ordre incorrect des conditions	10
Piège avec or	11
1.15 Gestion des cas limites	11
1.16 Approche structurée professionnelle	12
2 Mini-exercices de production	12
Exercice 1 — Prédiction	12
Exercice 2 — Production	12
Exercice 3 — Débogage	13
Synthèse	14
20 Questions Intelligentes — Flux Conditionnels	14
1) Différence entre and et &	15
Énoncé	15
Solution	15
2) Pourquoi l'ordre des elif est critique ?	15
Énoncé	15
Solution	15
3) Pourquoi éviter les if imbriqués excessifs ?	16
Énoncé	16
Solution	16

4) Évaluation booléenne implicite	17
Énoncé	17
Solution	17
5) Chaînage des comparaisons	17
Énoncé	17
Solution	17
6) Court-circuit logique (short-circuit)	18
Énoncé	18
Solution	18
7) Piège avec or	18
Énoncé	18
Solution	18
8) Comparaison de chaînes sensible à la casse	19
Énoncé	19
Solution	19
9) None vs 0	19
Énoncé	19
Solution	20
10) Comparaison avec is	20
Énoncé	20
Solution	20
11) Ternaire imbriqué : lisible ou dangereux ?	21
Énoncé	21
Solution	21
12) Évaluation de valeurs falsy	21
Énoncé	21
Solution	22
13) Priorité des opérateurs	22
Énoncé	22
Solution	22
14) Comparaison de flottants	23
Énoncé	23
Solution	23
15) Validation des entrées utilisateur	23
Énoncé	23
Solution	23

16) Simplification logique	24
Énoncé	24
Solution	24
17) Cas métier : contrôle d'accès	25
Énoncé	25
Solution	25
18) Erreur classique avec input	25
Énoncé	25
Solution	25
19) Division par zéro	26
Énoncé	26
Solution	26
20) Refactorisation avancée	26
Énoncé	26
Solution	27

Les flux conditionnels

Le support est propriété de **Bechir Bejaoui**

1 Présentation des flux conditionnels

1.1 Objectif des structures conditionnelles

Un programme professionnel ne se contente pas d'exécuter des instructions séquentiellement. Il doit :

- Prendre des décisions
- Contrôler l'exécution
- Gérer des cas métier réels
- Sécuriser des scénarios imprévus

Les flux conditionnels permettent d'orienter l'exécution selon des règles logiques.

1.2 Logique fondamentale

Un flux conditionnel repose sur :

- Une expression logique

- Une évaluation booléenne
- Une bifurcation d'exécution

Schéma conceptuel :

Début

- Évaluation d'une condition
- Exécution bloc A si vrai
- Exécution bloc B si faux

1.3 Le mot-clé if

Structure de base :

```
if condition:
    bloc_exécuté_si_vrai
```

Exemple minimal :

```
"""
La condition retourne True ou False.
Le bloc indenté s'exécute uniquement si True.
L'indentation définit la portée du bloc.
"""
```

```
age = 20
```

```
if age >= 18:
    print("Majeur")
```

1.4 Structure if / else

Permet de définir un comportement alternatif.

```
"""
Une seule branche s'exécute.
L'évaluation est exclusive.
"""
```

```
age = 16
```

```
if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

1.5 Structure if / elif / else

Permet de gérer plusieurs cas mutuellement exclusifs.

```
"""
Les conditions sont évaluées dans l'ordre.
Dès qu'une condition est vraie, les suivantes ne sont plus évaluées.
else est facultatif mais recommandé pour couvrir les cas restants.
"""

note = 14

if note < 10:
    print("Échec")
elif note < 12:
    print("Passable")
elif note < 14:
    print("Assez bien")
elif note < 16:
    print("Bien")
else:
    print("Très bien")
```

1.6 Expression conditionnelle (opérateur ternaire)

Forme compacte :

valeur_si_vrai if condition else valeur_si_faux

```
"""
Forme concise pour affectation simple.
À éviter pour les logiques complexes ou imbriquées.
"""
```

```
a = 5
b = 10
```

```
resultat = "A plus grand" if a > b else "B plus grand ou égal"
print(resultat)
```

Utilisation adaptée :

- Affectation simple
- Retour de fonction

- Expressions concises

À éviter :

- Imbrications multiples
 - Logique complexe
-

1.7 Opérateurs de comparaison

```
"""
Les six opérateurs de comparaison standard.
Chacun retourne True ou False.
"""

temperature = 35

print(temperature == 35)    # égalité
print(temperature != 30)    # différence
print(temperature > 30)     # supérieur
print(temperature >= 35)    # supérieur ou égal
print(temperature < 40)     # inférieur
print(temperature <= 35)    # inférieur ou égal

if temperature > 30:
    print("Alerte chaleur")
```

1.8 Opérateurs logiques

Permettent de combiner plusieurs conditions.

```
"""
and → toutes les conditions doivent être vraies
or  → au moins une condition doit être vraie
not → inversion logique
"""

age = 25
permis = True

if age >= 18 and permis:
    print("Autorisé à conduire")

if age < 18 or not permis:
    print("Non autorisé")
```

1.9 Évaluation booléenne implicite

Python considère comme False :

- 0
- None
- ""
- []
- {}
- False

```
"""
Python évalue directement la vérité d'une valeur.
Éviter les comparaisons explicites inutiles.
"""
```

```
panier = []

if panier:
    print("Panier non vide")
else:
    print("Panier vide")

# Bonne pratique
if panier:          # pythonique
    pass

if len(panier) > 0: # redondant
    pass
```

1.10 Chaînage des comparaisons

```
"""
Python permet de chaîner les comparaisons.
Plus lisible et plus sûr qu'une double condition and.
"""
```

```
age = 25

if 18 <= age < 65:
    print("Actif")
```

```
# Équivalent mais moins lisible
if age >= 18 and age < 65:
    print("Actif")
```

1.11 Court-circuit logique

```
"""
Avec and : si la première condition est False,
la seconde n'est pas évaluée.
Avec or  : si la première condition est True,
la seconde n'est pas évaluée.
Utile pour éviter des erreurs d'exécution.
"""

x = 0

if x != 0 and 10 / x > 1:
    print("Valide")
else:
    print("Division évitée grâce au court-circuit")
```

1.12 match / case — Python 3.10+

```
"""
match/case est la solution moderne pour remplacer
les longues chaînes if/elif sur une même variable.
Plus lisible et plus expressif.
"""

commande = "quitter"

match commande:
    case "démarrer":
        print("Application démarrée")
    case "arrêter":
        print("Application arrêtée")
    case "quitter":
        print("Au revoir")
    case _:
        print("Commande inconnue")
```


Avec conditions — guards

```
"""
Les guards permettent d'ajouter une condition
à l'intérieur d'un case.
"""
```

```
score = 85
```

```
match score:
    case s if s >= 90:
        print("Excellent")
    case s if s >= 70:
        print("Bien")
    case s if s >= 50:
        print("Passable")
    case _:
        print("Insuffisant")
```

1.13 Cas métier — Système de validation d'accès

```
"""
Un système doit autoriser un utilisateur si :
- Il est actif
- Son abonnement n'est pas expiré
- Son rôle est autorisé
Utilisation de in au lieu de multiples or.
"""
```

```
actif = True
abonnement_valide = True
role = "admin"
```

```
if actif and abonnement_valide and role in ["admin", "manager"]:
    print("Accès autorisé")
else:
    print("Accès refusé")
```

1.14 Pièges fréquents

Confusion entre = et ==

```
"""
= est une affectation.
```

```
== est une comparaison.  
Utiliser = dans un if provoque une SyntaxError.  
"""
```

```
age = 18
```

```
# Incorrect → SyntaxError  
# if age = 18:
```

```
# Correct  
if age == 18:  
    print("Exactement 18 ans")
```

Confusion entre & et and

```
"""  
and est un opérateur logique - court-circuit.  
& est un opérateur bit à bit - pas de court-circuit.  
Utiliser and dans les conditions logiques.  
"""
```

```
a = 5
```

```
# Correct  
if 0 < a < 10:  
    print("Dans l'intervalle")  
  
# Risqué  
if (0 < a) & (a < 10):  
    print("Fonctionne mais déconseillé")
```

Ordre incorrect des conditions

```
"""  
Une condition large placée en premier  
rend les conditions suivantes inaccessibles.  
"""
```

```
age = 25
```

```
# Incorrect - la seconde condition n'est jamais atteinte  
if age >= 0:  
    print("Age valide")
```

```

elif age >= 18:
    print("Majeur")      # jamais atteint

# Correct
if age >= 18:
    print("Majeur")
elif age >= 0:
    print("Mineur")

```

Piège avec or

```

"""
"manager" seul est toujours évalué comme True.
Il ne compare pas role à "manager".
"""

role = "user"

# Incorrect - toujours True
if role == "admin" or "manager":
    print("Toujours exécuté")

# Correct
if role == "admin" or role == "manager":
    print("Condition correcte")

# Pythonique
if role in ["admin", "manager"]:
    print("Condition optimisée")

```

1.15 Gestion des cas limites

```

"""
Toujours prévoir :
- Valeurs négatives
- Valeurs nulles
- Valeurs hors intervalle
"""

montant = 0

if montant < 0:
    print("Montant invalide")

```

```
elif montant == 0:
    print("Aucun paiement requis")
else:
    print("Paiement en cours :", montant)
```

1.16 Approche structurée professionnelle

Dans un projet réel :

- Centraliser la logique métier
 - Éviter les if imbriqués excessifs
 - Favoriser la lisibilité
 - Couvrir tous les cas
 - Tester les bornes
-

2 Mini-exercices de production

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
x = 5

print("A" if x > 10 else "B" if x > 3 else "C")

panier = []
if panier:
    print("Non vide")
else:
    print("Vide")

role = "user"
if role == "admin" or "manager":
    print("OK")
```

Exercice 2 — Production

Écrire un programme qui :

1. Demande un score entre 0 et 100
2. Valide que le score est dans l'intervalle $0 \leq \text{score} \leq 100$

3. Affiche la mention correspondante
4. Gère le cas d'un score invalide

Votre solution ici

Solution

```
"""
Validation d'un score et affichage de la mention.
"""

score = int(input("Score (0-100) : "))

if not (0 <= score <= 100):
    print("Score invalide")
elif score >= 90:
    print("Mention : Excellent")
elif score >= 70:
    print("Mention : Bien")
elif score >= 50:
    print("Mention : Passable")
else:
    print("Mention : Insuffisant")
```

Exercice 3 — Débogage

Ce code contient 3 erreurs. Les identifier et les corriger.

```
age = 20
permis = True

if age = 18:
    print("Exactement 18 ans")

role = "manager"
if role == "admin" or "manager":
    print("Accès autorisé")

solde = 0
if solde:
    print("Solde disponible")
elif solde < 0:
    print("Solde négatif")
else:
    print("Solde nul")
```

Solution

```

# Erreur 1 : = au lieu de == dans le if → SyntaxError
# Erreur 2 : or "manager" toujours True → utiliser role in [...]
# Erreur 3 : if solde évalué 0 comme False
#           → la condition elif solde < 0 n'est jamais atteinte pour solde=0

age      = 20
permis   = True

if age == 18:
    print("Exactement 18 ans")

role = "manager"
if role in ["admin", "manager"]:
    print("Accès autorisé")

solde = 0
if solde < 0:
    print("Solde négatif")
elif solde == 0:
    print("Solde nul")
else:
    print("Solde disponible :", solde)

```

Synthèse

- `if` / `elif` / `else` couvre les cas mutuellement exclusifs
 - L'ordre des `elif` est critique — du plus restrictif au plus large
 - L'évaluation implicite évite les comparaisons redondantes
 - Le chaînage `18 <= age < 65` est plus lisible qu'un double `and`
 - Le court-circuit `and` / `or` protège contre les erreurs d'exécution
 - `match/case` (Python 3.10+) remplace avantageusement les longues chaînes `if/elif`
 - Ne jamais écrire `role == "admin" or "manager"` — utiliser `in`
 - Toujours couvrir les cas limites : `None`, `0`, `""`, valeurs négatives
-

20 Questions Intelligentes — Flux Conditionnels

1) Différence entre and et &

Énoncé

Pourquoi utiliser and au lieu de & dans une condition logique ?

Solution

```
"""
Explication :
and est un opérateur logique avec court-circuit.
& est un opérateur bit à bit sans court-circuit.
Les deux ne sont pas équivalents.
"""

a = 5
b = 3

if a > 0 and b > 0:
    print("Condition logique valide")

if (a > 0) & (b > 0):
    print("Condition bit à bit exécutée")

print("Résultat and :", a > 0 and b > 0)
print("Résultat &   :", (a > 0) & (b > 0))
```

2) Pourquoi l'ordre des elif est critique ?

Énoncé

Que se passe-t-il si les conditions sont mal ordonnées ?

Solution

```
"""
Explication :
Les conditions sont évaluées de haut en bas.
Une condition large placée en premier peut rendre
les suivantes inaccessibles.
"""

age = 25
```

```

# Mauvaise structure
if age >= 0:
    print("Age valide")
elif age >= 18:
    print("Majeur")      # jamais atteint

# Structure correcte
if age >= 18:
    print("Majeur")
elif age >= 0:
    print("Mineur")

```

3) Pourquoi éviter les if imbriqués excessifs ?

Énoncé

Quel est le risque des structures profondément imbriquées ?

Solution

```

"""
Explication :
Lisibilité réduite.
Complexité accrue.
Risque d'erreurs logiques.
Préférer combiner avec and.
"""

age = 30
permis = True

# Version imbriquée
if age >= 18:
    if permis:
        print("Autorisé")

# Version simplifiée
if age >= 18 and permis:
    print("Autorisé (version claire)")

```

4) Évaluation booléenne implicite

Énoncé

Pourquoi `if liste:` est préférable à `if len(liste) > 0` ?

Solution

```
"""
Explication :
Python évalue directement la vérité d'une collection.
Une liste vide est False, une liste non vide est True.
"""

liste = [1, 2, 3]

if liste:
    print("Liste non vide")

if len(liste) > 0:
    print("Version équivalente mais moins pythonique")
```

5) Chaînage des comparaisons

Énoncé

Pourquoi `0 < x < 10` est préférable ?

Solution

```
"""
Explication :
Plus lisible et plus sûr.
Évite la répétition de la variable.
"""

x = 5

if 0 < x < 10:
    print("Dans l'intervalle")

# Équivalent mais moins lisible
if x > 0 and x < 10:
    print("Dans l'intervalle")
```

6) Court-circuit logique (short-circuit)

Énoncé

Pourquoi `and` peut empêcher une erreur ?

Solution

```
"""
Explication :
Si la première condition est False,
la seconde n'est pas évaluée.
Évite ici une ZeroDivisionError.
"""

x = 0

if x != 0 and 10 / x > 1:
    print("Valide")
else:
    print("Division évitée grâce au court-circuit")
```

7) Piège avec `or`

Énoncé

Pourquoi cette condition est toujours vraie ?

```
if role == "admin" or "manager":
```

Solution

```
"""
Explication :
"manager" est une chaîne non vide → toujours True.
Il ne compare pas role à "manager".
"""

role = "user"

# Incorrect - toujours True
if role == "admin" or "manager":
```

```

    print("Toujours exécuté (erreur)")

# Correct
if role == "admin" or role == "manager":
    print("Condition correcte")

# Pythonique
if role in ["admin", "manager"]:
    print("Condition optimisée")

```

8) Comparaison de chaînes sensible à la casse

Énoncé

Pourquoi certaines comparaisons échouent ?

Solution

```

"""
Explication :
Les chaînes sont sensibles à la casse.
Normaliser avec lower() ou casefold() avant de comparer.
"""

reponse = "Oui"

if reponse.lower() == "oui":
    print("Validation réussie")

# Sans normalisation
if reponse == "oui":
    print("Jamais affiché si l'utilisateur tape Oui")

```

9) None vs 0

Énoncé

Pourquoi confondre None et 0 est dangereux ?

Solution

```
"""
Explication :
None signifie absence de valeur.
0 est une valeur numérique valide.
Les deux sont falsy mais pour des raisons différentes.
"""

valeur = None

if valeur is None:
    print("Aucune valeur fournie")

valeur = 0

if valeur == 0:
    print("Valeur numérique zéro")

# Les deux sont falsy
if not None:
    print("None est falsy")

if not 0:
    print("0 est falsy")
```

10) Comparaison avec is

Énoncé

Pourquoi utiliser is pour None ?

Solution

```
"""
Explication :
None est un singleton - un seul objet en mémoire.
is compare l'identité, pas la valeur.
Plus sûr et plus expressif que == None.
"""

valeur = None

if valeur is None:
```

```
print("Comparaison correcte avec is")

if valeur is not None:
    print("Valeur présente")
```

11) Ternaire imbriqué : lisible ou dangereux ?

Énoncé

Quand éviter les ternaires imbriqués ?

Solution

```
"""
Explication :
Un ternaire imbriqué nuit à la lisibilité.
Préférer if/elif/else pour les logiques multiples.
"""

x = 5

# Difficile à lire
result = "A" if x > 10 else "B" if x > 3 else "C"
print(result)

# Version claire
if x > 10:
    result = "A"
elif x > 3:
    result = "B"
else:
    result = "C"

print(result)
```

12) Évaluation de valeurs falsy

Énoncé

Quelles valeurs Python évalue-t-il comme False ?

Solution

```
"""
Explication :
Python évalue comme False :
0, None, "", [], {}, set(), False.
Toutes les autres valeurs sont truthy.
"""

valeurs_falsy = [0, None, "", [], {}, set(), False]

for v in valeurs_falsy:
    if not v:
        print(f"{repr(v):10} → False")

print("\nValeurs truthy :")
for v in [1, "texte", [0], {"a": 1}]:
    if v:
        print(f"{repr(v):15} → True")
```

13) Priorité des opérateurs

Énoncé

Pourquoi ajouter des parenthèses améliore la sécurité ?

Solution

```
"""
Explication :
and a une priorité plus haute que or.
Les parenthèses rendent l'intention explicite
et évitent les erreurs subtiles.
"""

a = 5
b = 10

# Ambigu sans parenthèses
if a > 3 and b > 5 or a == 0:
    print("Résultat ambigu")

# Intention claire
if (a > 3 and b > 5) or a == 0:
```

```
print("Condition claire")
```

14) Comparaison de flottants

Énoncé

Pourquoi éviter `==` avec float ?

Solution

```
"""
Explication :
Les floats sont approximatifs en binaire.
Utiliser abs() ou math.isclose() pour comparer.
"""

import math

x = 0.1 + 0.2

# Dangereux
print(x == 0.3)

# Correct
if abs(x - 0.3) < 0.00001:
    print("Comparaison sécurisée avec abs()")

if math.isclose(x, 0.3):
    print("Comparaison sécurisée avec math.isclose()")
```

15) Validation des entrées utilisateur

Énoncé

Pourquoi toujours valider les entrées ?

Solution

```
"""
Explication :
Les entrées peuvent être invalides, négatives ou hors intervalle.
Toujours valider avant traitement.
"""
```

```

"""

montant = -10

if montant is None:
    print("Aucune valeur fournie")
elif not isinstance(montant, (int, float)):
    print("Type invalide")
elif montant < 0:
    print("Montant négatif invalide")
elif montant == 0:
    print("Aucun paiement requis")
else:
    print("Paiement valide :", montant)

```

16) Simplification logique

Énoncé

Pourquoi éviter les comparaisons inutiles ?

Solution

```

"""
Explication :
Une variable booléenne est déjà True ou False.
La comparer à True est redondant.
"""

flag = True

# Redondant
if flag == True:
    print("Redondant")

# Pythonique
if flag:
    print("Direct et lisible")

# Pour False
if not flag:
    print("Condition négative")

```

17) Cas métier : contrôle d'accès

Énoncé

Implémenter une validation robuste multi-critères.

Solution

```
"""
Explication :
Combiner plusieurs conditions métier.
Utiliser not pour les conditions négatives.
Utiliser in pour les listes de valeurs autorisées.
"""

actif          = True
role           = "manager"
solde_impaye   = False

if actif and not solde_impaye and role in ["admin", "manager"]:
    print("Accès accordé")
else:
    print("Accès refusé")
```

18) Erreur classique avec input

Énoncé

Pourquoi comparer une entrée texte à un nombre échoue ?

Solution

```
"""
Explication :
input() retourne toujours une chaîne.
La comparaison avec un entier retourne False.
Convertir avec int() ou float() avant comparaison.
"""

age_str = input("Age : ")

# Dangereux sans conversion
if age_str >= 18:    # TypeError
    print("Majeur")
```

```
# Correct
age = int(age_str)

if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

19) Division par zéro

Énoncé

Comment sécuriser une division dans un flux conditionnel ?

Solution

```
"""
Explication :
Toujours vérifier le diviseur avant la division.
Le court-circuit and évite l'évaluation si b == 0.
"""

a = 10
b = 0

if b != 0:
    print(a / b)
else:
    print("Division impossible")

# Version avec court-circuit
resultat = a / b if b != 0 else None
print("Résultat :", resultat)
```

20) Refactorisation avancée

Énoncé

Quand remplacer un long if/elif par un dictionnaire ?

Solution

```
"""
Explication :
Un dictionnaire de fonctions remplace avantageusement
une longue chaîne if/elif sur une même variable.
Plus extensible et plus lisible.
"""

def addition(a, b):
    return a + b

def soustraction(a, b):
    return a - b

def multiplication(a, b):
    return a * b

operations = {
    "+": addition,
    "-": soustraction,
    "*": multiplication
}

op = "+"
a = 5
b = 3

if op in operations:
    print(f"{a} {op} {b} = {operations[op](a, b)}")
else:
    print("Opération invalide")
```